

Cybernetics Oriented Programming (CYBOP)

GNU GPLv3

Christian Heller
<christian.heller@tuxtax.de>



Application knowledge

CYBOL

Control software

CYBOI

Operating system

Hardware

Computer

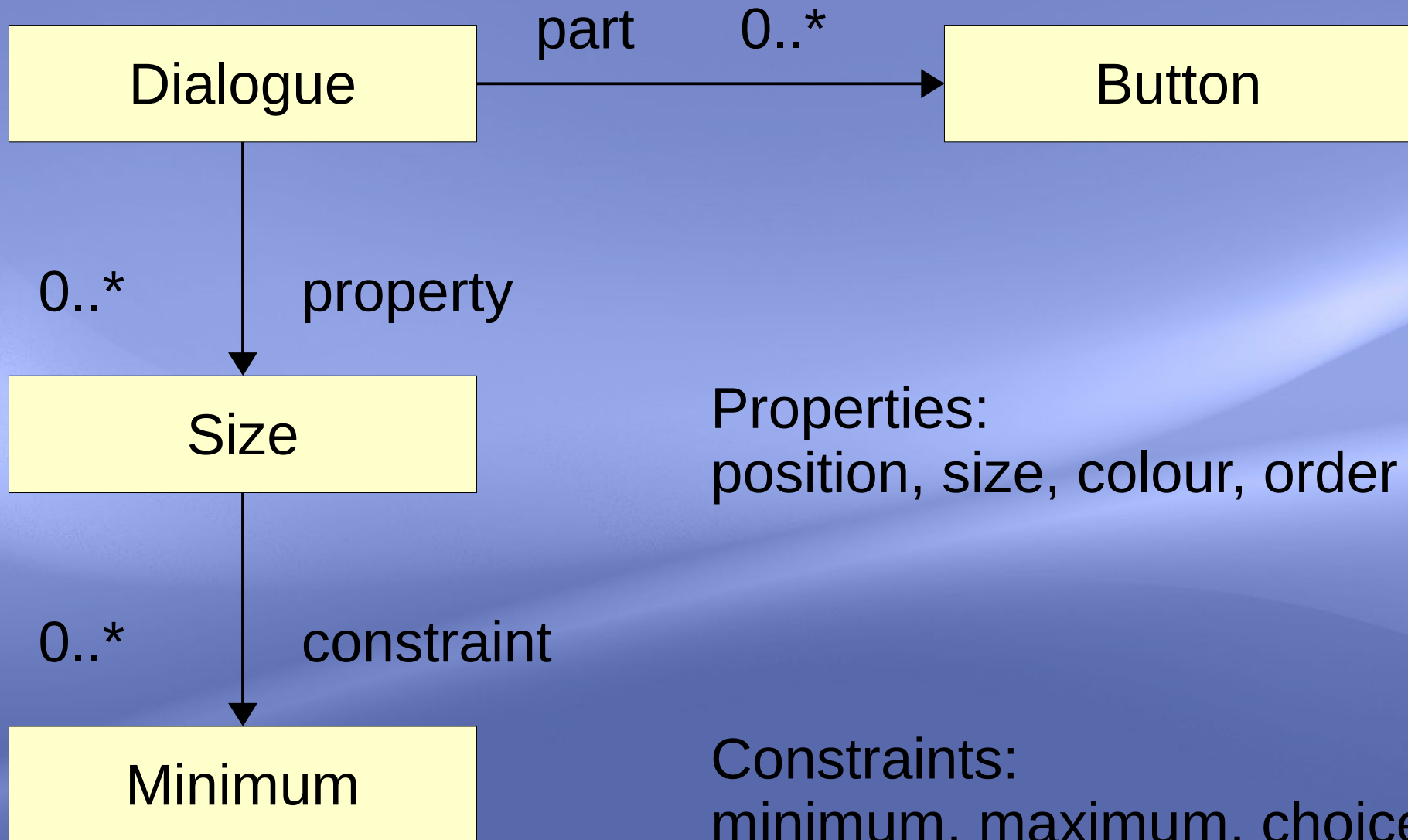
cyboi app/run.cybol

Criterion	Java world	CYBOP world
Language	Java	CYBOL (XML-based)
Interpreter	Java Virtual Machine (JVM)	CYBOI (written in C)
Theory	Object Oriented Programming (OOP)	CYBOP (knowledge schema)

Java Class

```
public class Dialogue {  
  
    // Structural attributes  
  
    private String title;  
    private Toolbar toolbar;  
    private Button button;  
  
    // Additional properties  
  
    private Point position;  
    private Dimension size;  
    private Color background;  
  
    // Methods  
  
    public void initialise(Object o);  
    public void process(Event e);  
}
```

Parts make up a whole



Properties:
position, size, colour, order

Constraints:
minimum, maximum, choice

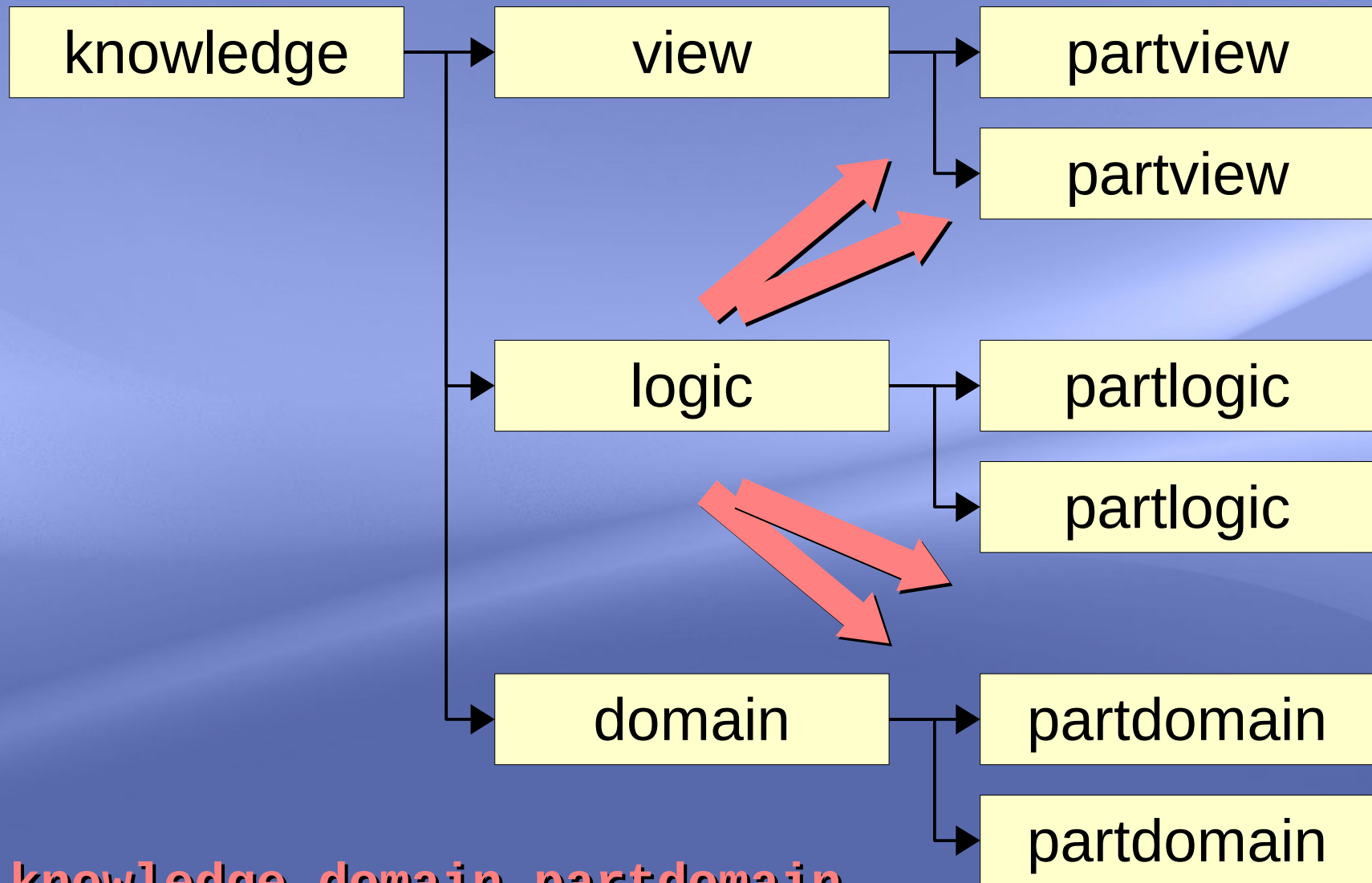
State Template "dialogue.cybol"

```
<model>
  <part name="title" channel="inline" format="character" model="Test"/>
  <part name="toolbar" channel="file" format="element/part" model="path/file.cybol"/>
  <part name="button" channel="file" format="element/part" model="path/button.cybol">
    <property name="position" channel="inline" format="number/integer" model="5,5"/>
    <property name="size" channel="inline" format="number/integer" model="600,400"/>
    <property name="background" channel="inline" format="colour/terminal" model="red"/>
  </part>
</model>
```

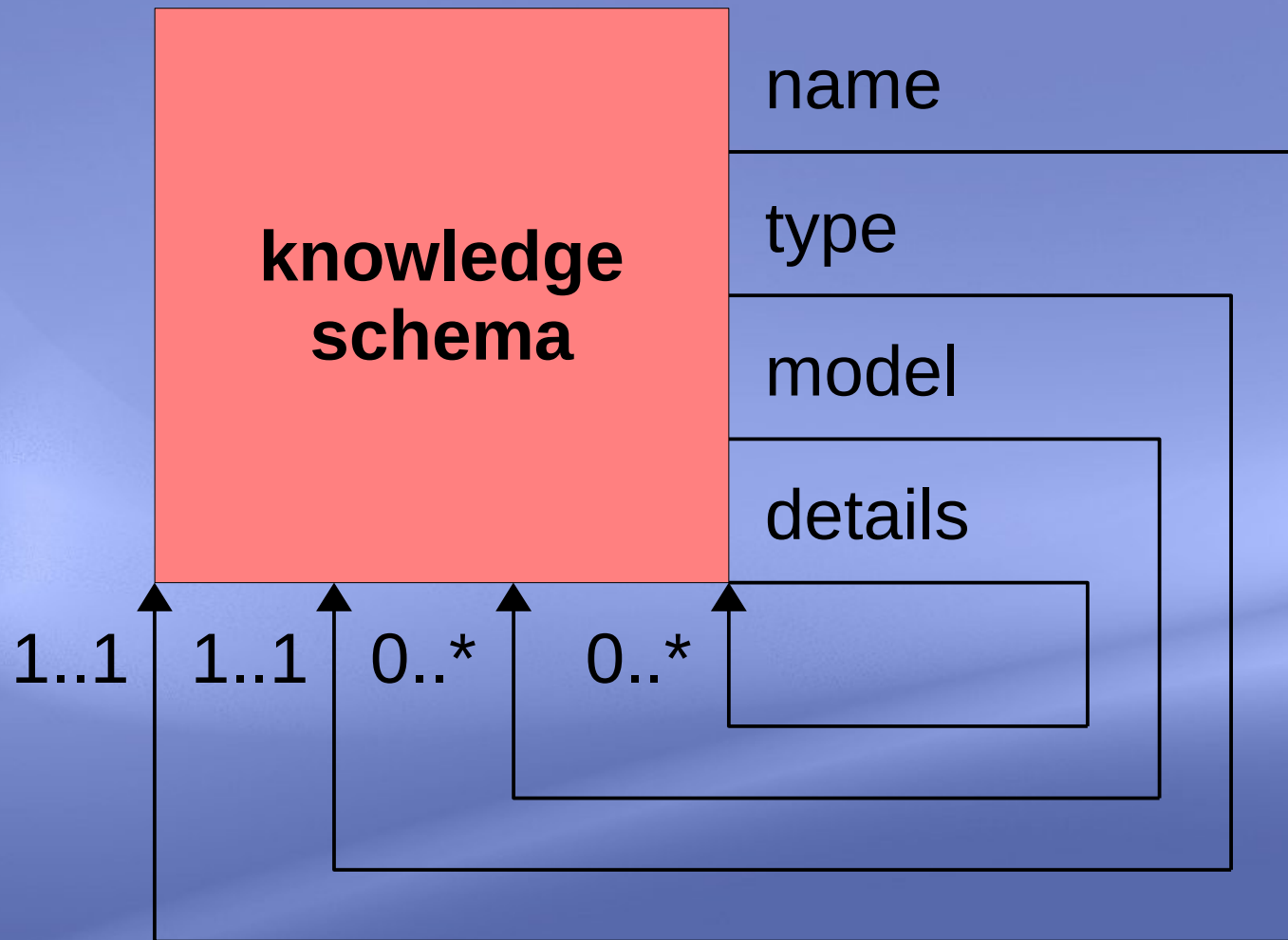
Logic Template "initialise.cybol"

```
<model>
  <part name="build_menu" channel="inline" format="path/knowledge" model=".logic.menu"/>
  <part name="build_panel" channel="inline" format="path/knowledge" model=".logic.panel"/>
  <part name="display" channel="inline" format="communicate/send" model="">
    <property name="channel" channel="inline" format="meta/channel" model="terminal"/>
    <property name="encoding" channel="inline" format="meta/encoding" model="utf-8"/>
    <property name="language" channel="inline" format="meta/language" model="message/tui"/>
    <property name="format" channel="inline" format="meta/format" model="element/part"/>
    <property name="message" channel="inline" format="path/knowledge" model=".dialogue"/>
    <property name="clear" channel="inline" format="logicvalue/boolean" model="true"/>
  </part>
</model>
```

knowledge.view.partview



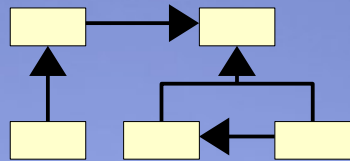
knowledge.domain.partdomain



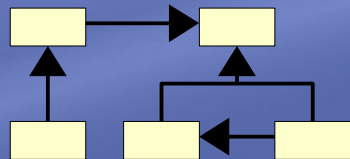
item
+ category
+ compound
= schema

relations unidirectional

traditional

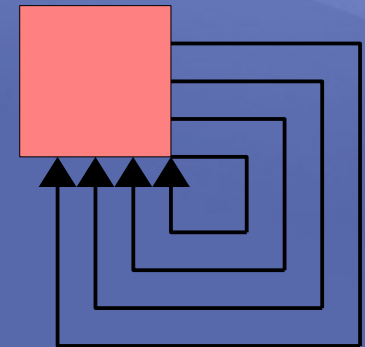
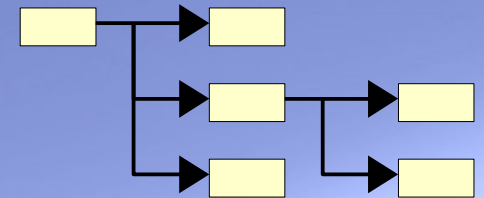


programme
structure



runtime
structure

cybop





traditional



cybop

models suffer from
complexity
strong coupling /
dependencies
inflexible
difficult to maintain

one schema as memory
structure
directed acyclic graph (tree)
easily extensible
long-life software system

declarative language cybol
programming theory cybop
interpreter cyboi

flexible and adaptable at runtime
easy transparent communication

prototype-based programming similar to JavaScript

application creation by domain experts without programming knowledge

identical syntax for state (structure) and logic (function)
uniform usage of elements/commands

oriented on human thinking (which OOP only partly is)
double hierarchy separating micro-/macrocosm structure from meta properties
name „cybernetics“, because many of its original ideas were taken from nature

all knowledge hierarchical in one application tree
comparable to representation of website as DOM tree

cybol: just 4 tags and 4 attributes
unnecessary data are just ignored without error

bundling of attributes and methods – coupling, no flexibility
reflective meta architectures – bidirectional dependencies
bidirectional dependencies – complexity, circular references
global / static data access – untraceable data manipulation
container inheritance – falsified container content

problems

inter-disciplinary ideas

biology – configuration by dna

psychology – item + category + compound

physics – space & time

philosophy – body & mind

characteristics

long-life software system due to strict separation of application knowledge and system software
less dependencies due to representation of structural- and meta data in form of a double hierarchy
flexible architecture due to unified schema as the only remaining static memory structure
clean knowledge model code due to tree structure with unidirectional relations
universal communication due to star-like (not layer-like) translation architecture
easy handling due to one syntax describing states (space) and algorithms (time) in the same way
superfluous implementation phase due to direct interpretation of knowledge

solved problems

falsified container content due to container inheritance
fragile base class problem due to polymorphism
untraceable data manipulation due to global data access
circular references due to bidirectional dependencies
bidirectional dependencies due to reflective meta architectures
strong coupling due to bundling of attributes and methods
difficult maintenance due to high complexity



although cybop
is free / open
source
software
(GPL), it may
well be used
for commercial
applications,
since
enterprise
knowledge
remains private

<http://www.cybop.org/>